



Secuencias básicas del lenguaje python

Clase

Aplicar el concepto de variable, tipos de datos fundamentales y expresiones aritméticas utilizando el lenguaje python para la creación de una rutina de baja complejidad. Codificar una rutina de baja complejidad en lenguaje python en el entorno de visual studio code.

{desafío}
latam_

- Unidad 1: El lenguaje Python
- Unidad 2: Sentencias básicas del lenguaje Python
- Unidad 3: Sentencias condicionales
- Unidad 4: Funciones y módulos
- Unidad 5: Estructura de datos en Python
- Unidad 6: Secuencias iterativas
- Unidad 7: Conceptos básicos de orientación a objeto en Python



Te encuentras aquí



¿Qué aprenderás en esta sesión?

- *Reconoce el concepto de variable identificando los tipos de datos fundamentales de acuerdo al lenguaje python.*
- *Elabora expresiones aritméticas para realizar cálculos de acuerdo al lenguaje python.*
- *Utiliza la entrada y salida estándar de python para el intercambio de datos con la rutina.*
- *Ejecuta un script con instrucciones python desde la consola.*

¿Qué sabes acerca de los
tipos de datos?



`/* Entrada y salida de datos */`

Impresión en consola

¿Cómo mostramos información? (*print*)

La función `print()` es nuestra forma de hacer que Python nos muestre información. Es como pedirle a Python que "hable" con nosotros.

```
# Mostrando un mensaje simple
print("¡Hola! Bienvenido a Python")

# Mostrando varios elementos
print("El resultado es:", 42)

# Mostrando en múltiples líneas
print("Primera línea")
print("Segunda línea")
```

Entrada de datos en consola

¿Cómo recibimos información? (input)

La función `input()` nos permite "preguntar" algo al usuario y recibir su respuesta.

```
# Preguntando al usuario
respuesta = input("¿Cómo te llamas? ")
print("¡Hola,", respuesta, "!")
```

¡Manos a la obra! Primer diálogo



Primer diálogo

1. Abre tu editor y crea un archivo `dialogo.py`
2. Escribe este código:

```
# Saludamos al usuario
print("¡Hola! Vamos a conocernos")

# Hacemos algunas preguntas
nombre = input("¿Cómo te llamas? ")
color = input("¿Cuál es tu color favorito? ")

# Respondemos usando la información
print("¡Mucho gusto,", nombre, "!")
print("El", color, "es un color muy bonito")
```



Primer diálogo

Cómo ejecutar un script Python desde la consola

3. Ejecuta el programa y Pruébalo.
4. Experimenta cambiando las preguntas y respuestas.

Cómo ejecutar un script Python desde la consola

- Guarda tu archivo Python (por ejemplo: dialogo.py).
- Abre una terminal o la terminal integrada de VS Code.
- Navega hasta la carpeta donde guardaste el archivo:
 - `cd ruta/de/tu/proyecto`
- Ejecuta el script:
 - `python dialogo.py`
- Observa la salida en la consola.



/* Variables */

¿Qué es una variable?

Se utiliza para guardar el resultado de operaciones o almacenar valores para ser reutilizados en el futuro. Podemos entender las variables como contenedores.

Una variable se compone de:

- Un nombre
- Un valor

Consideraciones

de variables

- El nombre de una variable comienza con minúscula.
- Si se requiere utilizar más de una palabra, éstas deben estar unidas por un guión bajo o underscore; a esto se le conoce como `snake_case`.
- Los nombres de las variables no pueden tener espacios ni comenzar con un número.
- Es importante utilizar nombres que sean representativos del valor que se quiere almacenar.

Por ejemplo, si queremos crear una variable que almacene la cantidad de alumnos de un curso, su nombre podría ser **`cant_alumnos`**.

Tipos de datos

de una variable

Una variable poseerá un tipo de dato asociado, según el valor que se le asigne, donde los tipos de datos pueden ser, entre otros, **integer**, **string**, **float** o **bool**.

Python dispone de la función **type()** para poder identificar el tipo de dato de una variable:

```
entero = 2
decimal = -6.5
texto = "Hola Mundo"

print(type(entero)) # int
print(type(decimal)) # float
print(type(texto)) # str
```

Es importante señalar que la función `type` se aplica a la variable, y no al `print()`.

Ejecutar `type(print(entero))` devolverá un tipo `None`, ya que `print` no es un tipo de dato, sino una función para imprimir en pantalla valores o información.



Manipulando variables

- Es posible trabajar con variables previamente definidas. Estas variables contendrán el valor asignado a lo largo de la ejecución de todo el código o hasta que este sea reasignado.
- Una variable de un tipo específico conserva todas las propiedades de su tipo de dato original.

```
a = 2 # se asigna el valor 2 a la variable a
a = a + 1 # se suma 1 al valor de a y se reasigna
print(a) # se muestra el resultado final
```

3

{desafío}
latam_

```
nombre = "Carlos"
# se asigna el valor Carlos a la variable nombre
apellido = "Santana"
# se asigna el valor Santana a la variable
apellido
```

```
print(nombre + " " + apellido)
# se muestra el resultado final
```

"Carlos Santana"

Transformando los datos

Interpolación

Permite introducir una variable, un dato o una operación válida dentro un String.

```
nombre = 'Carlos'
apellido = 'Santana'
# Interpolación
print("Mi nombre es {} {}".format(nombre,
apellido))
```

```
nombre = 'Carlos'
apellido = 'Santana'
# Interpolación
print(f"Mi nombre es {nombre} {apellido}")
```

Mi nombre es Carlos Santana

{desafío}
latam_

Precisión de datos

Permite controlar la precisión de los decimales de manera muy sencilla.

```
# Operación que arroja varios decimales
print(f'El resultado es {1/9}')
```

El resultado es 0.1111111111111111

```
# Muestro sólo 2 decimales
print(f'El resultado es {1/9:.2f}')
```

El resultado es 0.11

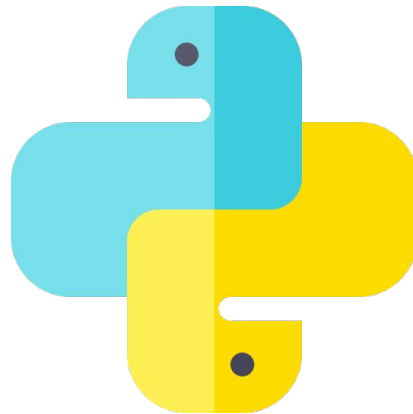
`/* Tipos de datos */`

Tipos de datos nativos en Python

Son distintas formas de mostrar información en el lenguaje.

En Python son:

- Valores numéricos
- Strings



Valores numéricos

Integer

```
# estos son valores integer
print(27)
print(-5)
print(0)
print(3 + 7)
print(31 - 7)
print(4 * 8)
```

Floats

```
# estos son valores float
print(27.6)
print(-5.9)
print(0.0)
print(3.2 + 7.456)
print(31.5 - 7.0)
print(4.190 * 8.12)
print(9.0 / 8.1)
```

- El separador decimal en Python es un “.”
- Cualquier operación de división entre integers, en Python el resultado será un float incluso si el resultado resulta en un valor entero. Ejemplo: `5.0 // 2` es `2.0`



Es posible realizar operaciones matemáticas combinando integers y float de manera válida como se muestra a continuación:

```
print(3.5 + 12)  
print(20 * 45.6)
```



Strings

- Son elementos encerrados entre comilla simple (') o comilla doble (").
- Para definirse como un string válido, la comilla al inicio y al final debe ser la misma.
- Normalmente, un string es un texto, o una etiqueta como nuestros nombres de pila, pero también podría contener caracteres numéricos, sin embargo no se podrán realizar operaciones matemáticas con estos últimos.

```
# estos son valores string
print('hola')
print("150")
print('este es un texto más largo')
```

Strings

*Las operaciones matemáticas entre un valor numérico y un string no son válidas.
Pero los strings sí tienen otras operaciones válidas.*

Concatenación

```
# Concatenación
print("Carlos" + "Santana") # CarlosSantana
print("Carlos " + "Santana") # Carlos Santana
```

```
# Concatenación
print("15" + "23") # esto resulta en 1523
```

Duplicación

```
# Duplicación
print(3 * "Carlos") # CarlosCarlosCarlos
print(5 * "12") # 1212121212
```


Valores booleanos

- Son valores lógicos que pueden tomar sólo dos valores:
 - True
 - False
- Estos valores son el resultado de alguna prueba lógica o de una asignación directa; donde este tipo de dato se revisará en detalle al momento de aprender Control de Flujo.

¡Manos a la obra!
Presentándome con python



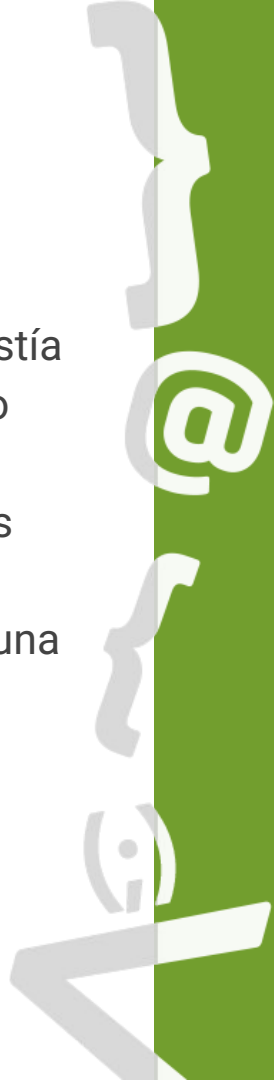
Presentándome con Python

Ejercicio guiado

Un gran conferencista, que siempre tenía que presentarse ante el público que asistía a sus conferencias, debía siempre dar una pequeña reseña de su vida, para que lo pudieran conocer. Hacer esto cada vez, lo aburrió y decidió pedirte que pudieras automatizar esta tarea, ya que lo bueno de la programación es automatizar tareas que son tediosas o que son propensas a error. Para esto, el conferencista te muestra una presentación antigua de una sus conferencias y te solicita elaborar una solución que permita modificarla levemente pero de manera fácil y rápida:

```
Mi nombre es Bill, tengo 52 años y me desempeño como CEO en Microsoft.  
Soy Creativo, Apasionado y Visionario.
```

```
Mi pasatiempo es jugar Golf y me gustaría poder jubilarme pronto.
```



Presentándome con Python

Ejercicio guiado

Entendamos el problema, esta es una plantilla, por lo tanto, podríamos determinar qué aspectos de la plantilla podrían cambiar. Algunos de estos aspectos son:

Nombre, Edad, Ocupación, Lugar de Trabajo, Características, Pasatiempos, Algo que hacer.

Al entender esto, quizás podríamos decir que estos son aspectos variables, que podrían cambiar en el tiempo o podrían ser modificados dependiendo de la persona.



Presentándome con Python

Ejercicio guiado

Entonces, hagamos un código que solicite estas variables, **presentador.py**:

```
nombre = input('Ingrese su Nombre: ')\nedad = input('Ingrese Edad: ')\nocupacion = input('Ingrese Ocupación: ')\nlugar = input('En dónde?: ')\ncaracteristica_1 = input('Ingrese 3 características:\n1. > ')\ncaracteristica_2 = input('2. > ')\ncaracteristica_3 = input('3. > ')\npasatiempo = input('Cuál es tu pasatiempo: ')\nhacer = input('¿Que quieres hacer? ')
```



Presentándome con Python

Ejercicio guiado

La idea es poder utilizarlas para construir el texto deseado, para ello, podríamos utilizar lo recién aprendido acerca de Interpolación. En este caso utilizaremos el string multilínea para poder escribir el texto de manera más ordenada:

```
print(f'''  
  
Mi nombre es {nombre}, tengo {edad} años y me desempeño como {ocupacion} en {lugar}.  
Soy una persona {caracteristica_1}, {caracteristica_2} y {caracteristica_3}.  
  
Mi pasatiempo es {pasatiempo} y me gustaría poder {hacer}.''' )
```



Presentándome con Python

Ejercicio guiado

Ahora, ejecutemos el programa:

```
Ingrese su Nombre: Bill
Ingrese Edad: 52
Ingrese Ocupación: CEO
En dónde?: Microsoft
Ingrese 3 características:
1. > Creativo
2. > Apasionado
3. > Visionario
Cuál es tu pasatiempo: jugar Golf
¿Que quieres hacer? jubilarme pronto

Mi nombre es Bill, tengo 52 años y me desempeño como CEO en Microsoft.
Soy una persona Creativo, Apasionado y Visionario.

Mi pasatiempo es jugar Golf y me gustaría poder jubilarme pronto.
```

Como se puede ver, se logró generar un programa en Python muy sencillo que permitirá generar presentaciones de manera muy rápida cambiando sólo algunos parámetros.



/* Expresiones aritméticas */

Expresiones aritméticas

En Python, las expresiones aritméticas son símbolos que se utilizan para realizar operaciones matemáticas entre valores numéricos.

Operador	Nombre	Ejemplo	Resultado
+	Suma	5+2	7
-	Resta	5-2	3
*	Multiplicación	5*2	10
/	División decimal	5/2	2.5
//	División entero	5//2	2
%	Módulo (resto de división)	5%2	1
**	Potencia (Exponenciación)	5**2	25

No olvidar que el módulo (resto) se representa con %

El módulo es para saber si la división es exacta o con decimales

El operador módulo (%) calcula el resto de una división entera.

Para entenderlo correctamente:

- En una división entera, nos preguntamos cuántas veces cabe el divisor dentro del dividendo.
- Luego, identificamos cuánto sobra después de tomar esas divisiones completas.
- Ese sobrante es el módulo.

Ejemplo correcto

División:

$$9 \div 5$$

1. **5 cabe 1 vez en 9** → porque $1 \times 5 = 5$
2. **Resto:**
 $9 - 5 = 4$

Por lo tanto:

- $9 // 5 = 1$ (cociente entero)
- $9 \% 5 = 4$ (resto)

¡Manos a la obra! Calculadora de descuentos



Calculadora de descuentos

Ejercicio guiado

Una tienda necesita calcular rápidamente el precio final de los productos que tienen descuento. El vendedor debe ingresar el precio original y el porcentaje de descuento, y el programa debe mostrar diferentes cálculos.

1. Crear el archivo calculadora_descuento.py
2. Definir las variables.

```
precio_original = input("Ingrese el precio del producto: $")
descuento = input("Ingrese el porcentaje de descuento (sin el %): ")

# Probar ejecutar y mostrar que recibimos Los datos
print("Precio ingresado:", precio_original)
print("Descuento ingresado:", descuento)
```



Calculadora de descuentos

Ejercicio guiado

3. Convertir los datos.

```
precio = float(precio_original) # Convertimos a decimal
porcentaje = float(descuento)   # Convertimos a decimal

# Mostrar el tipo de datos antes y después
print("Tipo de precio_original:", type(precio_original))
print("Tipo de precio:", type(precio))
```



Calculadora de descuentos

Ejercicio guiado

4. Realizar los cálculos.

```
# Cálculos
valor_descuento = precio * (porcentaje / 100)
precio_final = precio - valor_descuento
iva = precio_final * 0.19
total_con_iva = precio_final + iva
```



Calculadora de descuentos

Ejercicio guiado

5. Formatear la salida de datos.

```
print(f"\n=== Resumen de la compra ===")
print(f"Precio original: ${precio:.2f}")
print(f"Descuento ({porcentaje}%): ${valor_descuento:.2f}")
print(f"Precio con descuento: ${precio_final:.2f}")
print(f"IVA (19%): ${iva:.2f}")
print(f"Total a pagar: ${total_con_iva:.2f}")
```



`/* Conversiones de tipo */`

Conversiones de tipo

Imagina que tienes una calculadora. Esta calculadora solo puede trabajar con números, no con palabras.

Cuando le pides información a Python usando `input()`, es como si alguien te escribiera en un papel. Aunque escribir números, siguen siendo texto (como una nota escrita), no hay números que puedas usar en la calculadora.

- **int()** : Convierte un entero
- **float()** : Convierte un decimal
- **str()** : Convierte un texto

```
edad = int("25")  
altura = float("1.75")  
texto = str(42)
```

Resumen

- En esta sesión, exploramos las herramientas fundamentales para crear programas básicos.
- Aprendimos a interactuar con el usuario mediante las funciones `print()` para mostrar información e `input()` para recibir datos.
- Estudiamos el concepto de variables como contenedores de información y los tres tipos de datos fundamentales: enteros (`int`), decimales (`float`) y texto (`str`).
- Un aspecto crucial fue la conversión entre tipos de datos, necesaria especialmente porque `input()` siempre devuelve texto. Esto es fundamental para realizar operaciones matemáticas básicas como suma, resta, multiplicación y división, que solo funcionan con datos del mismo tipo.
- Los errores más comunes incluyen olvidar convertir datos antes de operar con ellos y usar variables sin inicializar. Para evitarlos, es importante seguir buenas prácticas como usar nombres descriptivos para las variables y probar el código con diferentes entradas.
- Estos conceptos básicos forman la base necesaria para crear programas más complejos y avanzar en el aprendizaje de Python.

¿Cuáles son los tipos de
datos nativos en Python?



¿De qué manera se permite
al usuario ingresar datos
en Python de manera
interactiva?



{desafío}
latam_

*Academia de
talentos digitales*

